

MORNING-Scabbard

v1.0 - Algorithm Specification

Supriya Adhikary	Songqiao Cui	Debayan Das	Abhinava De
Anindya Ganguly	Harry Hart	Angshuman Karmakar	Suparna Kundu
Chaoyun Li	Prajna Mahadev	Puja Mondal	Quinten Norga
	Thales B. Paiva	Debranjana Pal	

Website: <https://sites.google.com/view/scabbard>

Date: 30th June 2026

Contents

1	Algorithm Specification and Description	6
1.1	Preliminaries and Notation	6
1.2	Specification of Scabbard.PKE	10
1.3	Specification of Scabbard.KEM	11
1.4	Scabbard Parameter Sets	12
1.5	Design Rationale	13
1.6	Features	15
2	Performance Evaluation	16
2.1	Analysis, Considerations and Trade-offs	16
2.2	Transmission and Storage Cost	16
2.3	Performance of Scabbard on Intel CPUs	16
2.3.1	Reference implementation	16
2.3.2	AVX2 implementation	17
2.4	Performance of Scabbard on ARM Cortex-M4 MCUs	17
3	Security Analysis	19
3.1	Theoretical Security	19
3.1.1	Correctness	21
3.1.2	IND-CCA security proof of Scabbard in the ROM	21
3.1.3	IND-CCA security proof of Scabbard in the QROM	21
3.2	Practical Security	22
3.2.1	Attacks against the underlying MLWR problem	22
3.2.2	Weighted Primal Attack	22
3.2.3	Weighted Dual Attack	22
3.2.4	Core-SVP hardness of Scabbard	23
3.2.5	Beyond core-SVP hardness	23
3.2.6	Attacks using decryption failures	24
3.3	Additional Security Properties	24
3.3.1	Forward Secrecy	24
3.3.2	Side-Channel Attack Resilience	25
3.3.3	Multi-key Attack Resilience	25
3.3.4	(Temporary) Key Misuse Resilience	26
4	Advantages and Limitations	28
4.1	Advantages	28
4.2	Limitations	28

List of Tables

1	Proposed parameter sets for Scabbard	12
2	Key and ciphertext sizes (in bytes) for all Scabbard parameter sets	16
3	Cycle counts of the C reference implementation for all parameter sets of Scabbard , obtained on one core of an 13th Gen Intel Core i7.	17
4	Cycle counts of the AVX2 reference implementation for all parameter sets of Scabbard , obtained on one core of 13th Gen Intel Core i7.	17
5	Cycle counts of the optimized implementations for all parameter sets of Scabbard on cortex-m4.	17
6	Peak stack usage of the optimized implementations for all parameter sets of Scabbard on cortex-m4.	18
7	Summary of security and correctness of Scabbard	19
8	Details of classical and quantum Core-SVP hardness of proposed Scabbard parameter sets. Beyond Core-SVP, the estimated cost of classical attacks in terms of gates and bit memory requirements.	23

List of Algorithms

1	$\text{PolyToBytes}_{\epsilon_k}(f)$	7
2	$\text{BytesToPoly}_{\epsilon_k}(B)$	7
3	$\text{SampleSecretCBD}(\sigma)$	8
4	$\text{SampleMatrixUD}(\rho)$	8
5	$\text{MinalB2.Encode}(\mathbf{m})$	9
6	$\text{MinalB2.Decode}(\mathbf{c})$	9
7	$\text{EncodeMessage}(\mathbf{m})$	9
8	$\text{DecodeMessage}(\mathbf{c})$	10
9	$\text{Scabbard.PKE.KeyGen}()$	10
10	$\text{Scabbard.PKE.Encrypt}(\text{pk}, m, r)$	11
11	$\text{Scabbard.PKE.Decrypt}(\text{sk}, c)$	11
12	$\text{Scabbard.KEM.KeyGen}()$	11
13	$\text{Scabbard.KEM.Encaps}(\overline{\text{pk}})$	12
14	$\text{Scabbard.KEM.Decaps}(\overline{\text{sk}}, c)$	12

Acronyms

CCA	Chosen-Ciphertext Attack.
CPA	Chosen-Plaintext Attack.
CPU	Central Processing Unit.
CSS	Classical Security Strength.
DFP	Decryption Failure Probability.
DSP	Digital Signal Processing.
FPGA	Field-Programmable Gate Array.
HSM	Hardware Security Module.
HW	Hardware.
ICCS	Institute of Commercial Cryptography Standards.
IoT	Internet of Things.
KEM	Key-Encapsulation Mechanism.
LWE	Learning with Errors.
LWR	Learning with Rounding.
MCU	Micro Controller Unit.
MLWE	Module Learning with Errors.
MLWR	Module Learning with Rounding.
NTT	Number Theoretic Transform.
PKE	Public-Key Encryption.
PQC	Post-Quantum Cryptography.
PRF	Pseudorandom Function.
QROM	Quantum Random Oracle Model.
QSS	Quantum Security Strength.
RBG	Random Bit Generator.
ROM	Random Oracle Model.
SCA	Side-Channel Analysis.
SW	Software.
UD	Uniform Distribution.
XOF	Extendable-Output Function.

Mathematical Symbols

n	Denotes the ring dimension, a power-of-two integer (see Section 1.4).
$\mathbb{B}^k$	A length- k array of 8-bit integers (bytes).
$\mathbb{Z}_m$	The ring of integers modulo m . Throughout this document, the moduli are power-of-two integers (see Section 1.4).
$f, \mathbf{v}, \mathbf{M}$	Throughout this document, polynomials are denoted through lower-case letters, vectors (of polynomials) through bold, lower-case letters and matrices (of polynomials) through bold, upper-case letters.
$\mathbf{v}^\top, \mathbf{M}^\top$	The transpose of a (row/column) vector $\mathbf{v}$ and a matrix $\mathbf{M}$.
$R_{m,n}$	The quotient ring $\mathbb{Z}_m[x]/(x^n + 1)$ consisting of polynomials of the form $f = f_0 + \dots + f_{n-1}X^{n-1}$, with $f_j \in \mathbb{Z}_m$ ($\forall j$). $R_{m,n}^\ell$ denotes the set of ℓ -dimensional vectors over $R_{m,n}$, whereas $R_{m,n}^{\ell \times k}$ denotes the ring of $(\ell \times k)$ -matrices over $R_{m,n}$.
$ A $	The absolute value of A , if A is a number. If A is an array, this denotes its length.
$A \parallel B$	The concatenation of two arrays/strings.
$A[i]$	The i -th entry in an array A . Throughout this document, all indexing starts at zero.
$\mathbf{v}[i], \mathbf{M}[i][j]$	The i -th entry in a polynomial vector $\mathbf{v}$ or the entry in the i -th row and j -th column of a matrix of polynomials $\mathbf{M}$. Furthermore, the c -th coefficient can be accessed as $\mathbf{v}[i][c]$ or $\mathbf{M}[i][j][c]$.
$A[k : m]$	The selection of the elements (e.g. $A[k], A[k + 1], \dots, A[m - 1]$) of an array A .
$:=$	The left-hand side is defined by the expression on the right-hand side.
$\ll, \gg$	Bitwise left/right shift operator. Throughout this document, all operators are extended to polynomials and vectors of polynomials by applying the operator coefficient-wise..
$\lceil \cdot \rceil$	The ceiling operator.
$\lfloor \cdot \rfloor$	The flooring operator.
$\lceil \cdot \rceil$	The rounding to the nearest integer.
$\oplus$	Boolean XOR.

1 Algorithm Specification and Description

Scabbard is an IND-CCA2-secure key-encapsulation mechanism (KEM) based on the presumed hardness of the module learning with rounding (MLWR) problem [7, 27]. The **Scabbard** scheme is constructed in two steps: (1) an IND-CPA-secure public-key encryption (PKE) scheme is constructed based on the MLWR problem; (2) the IND-CCA2-secure KEM **Scabbard** is then obtained by applying a slightly modified Fujisaki–Okamoto (FO) transform [17] to the IND-CPA-secure PKE scheme. A preliminary version of the design of this scheme was laid out in [28, 26].

Scabbard was designed with simplicity in mind, enabling efficient and compact implementations on both hardware (HW) and software (SW) platforms. Core design choices of **Scabbard** include the use of power-of-two integer moduli, avoiding explicit modular reduction and rejection sampling; and the use of a module structure, allowing to scale between security levels without affecting the core arithmetic.

1.1 Preliminaries and Notation

Scabbard is parametrized using:

- n, ℓ : the ring dimension $n \in \{64, 128, 256\}$ of the polynomial ring $\mathbb{Z}_m[x]/(x^n + 1)$ with modulus m and the rank $\ell \in \{8, 9\}$ of the module. Increasing the degree/rank increases the security.
- q, p, t : the moduli of **Scabbard** are chosen to be power-of-two integers with $t < p < q$. More specifically, they are parametrized as $q = 2^{\epsilon_q}$, $p = 2^{\epsilon_p}$ and $t = 2^{\epsilon_t}$ (see Section 1.4).
- κ : the size of seeds for pseudorandomly deriving matrices and vectors (in bytes) (see Section 1.4).

Rounding In **Scabbard**, instead of explicitly performing the rounding operation, simple bit shifts are used. For this, **Scabbard** uses a constant vector $\mathbf{h} \in R_{q,n}^\ell$, in which each polynomial coefficient is equal to $2^{\epsilon_q - \epsilon_p - 1}$.

Symmetric Primitives **Scabbard** relies on cryptographic pseudorandom functions (PRF) and extendable output functions (XOF). Below we define the specific instantiations/wrappers that are used in **Scabbard**, where the specified input/output lengths are in number of bytes (byte array length).

Each of the defined PRF primitives are parametrized, each taking in a fixed-length input byte array M and parameter(s), and producing one fixed-length output byte array r . The function **PRF1** is parameterized by $n \in \{64, 128, 256\}$ and $\kappa \in \{16, 32, 64\}$. It takes in a $(\kappa + 1)$ -byte input, and produces a $(n/2)$ -byte output array and is denoted as:

$$\text{PRF1}_{\kappa,n}(M) : \mathbb{B}^{\kappa+1} \rightarrow \mathbb{B}^{n/2} \quad \text{with } \kappa \in \{16, 32, 64\} \text{ and } n \in \{64, 128, 256\}. \quad (1.1)$$

Similarly, the function **PRF2** takes as input a $(\kappa + 2)$ -byte string, and outputs an $(n\epsilon_q/8)$ -byte string, denoted as

$$\text{PRF2}_{\kappa,n,\epsilon_q}(M) : \mathbb{B}^{\kappa+2} \rightarrow \mathbb{B}^{n\epsilon_q/8} \quad \text{with } \kappa \in \{16, 32, 64\}, n \in \{64, 128, 256\}, \epsilon_q \in \{13, 14\}. \quad (1.2)$$

Extendable-output functions (XOF) can absorb one variable-length input and produce one variable-length output. For an input byte array M , the **XOF** produces a byte array r of k bytes and is denoted as:

$$r \leftarrow \text{XOF}(M, k) \quad (1.3)$$

The PRF primitives, which are defined above, can be instantiated using an **XOF**, for example:

$$\text{PRF1}_n(M) := \text{XOF}(M, n/2) \quad (1.4)$$

Randomness Generation In pseudocode, $s \xleftarrow{\$} \mathbb{B}^k$ denotes that s (length- k array) is freshly sampled from a uniformly random distribution, generated by a random number generator (RNG).

Bits, Bytes, Polynomials and Conversions Byte arrays can be converted to bit arrays through the **BytesToBits** routine, which converts an array of k bytes to an array of $8k$ bits. The bit b_i at index i in the output bit array is computed using the input byte array B as follows:

$$\text{BytesToBits: } b_i \leftarrow \left(\frac{B_{\lfloor i/8 \rfloor}}{2^{i \bmod 8}} \bmod 2 \right) \quad \forall i \in \{0, \dots, 8k-1\} \quad (1.5)$$

Similarly, the reverse operation **BitsToBytes**, converts a bit array to a byte array. Each output byte B_i is computed using the input bit array b as follows:

$$\text{BitsToBytes: } B_i \leftarrow \sum_{j=0}^7 (b_{8i+j} \cdot 2^j) \quad \forall i \in \{0, \dots, k-1\} \quad (1.6)$$

Furthermore, we use **PolyToBytes** and its inverse **BytesToPoly** (Algorithm 1 & 2) to convert between polynomial representations (arrays of ϵ_k -bit integers) and byte arrays.

Algorithm 1 **PolyToBytes** $_{\epsilon_k}(f)$

Converts an array of integers modulo 2^{ϵ_k} to a byte array.

Input: polynomial coefficients $\{f_0, \dots, f_{n-1}\} \in \mathbb{Z}_{2^{\epsilon_k}}$

Output: byte array $B \in \mathbb{B}^{\epsilon_k n/8}$

```

1: for ( $i \leftarrow 0$ ;  $i < n$ ;  $i = i + 1$ )
2:   for ( $j \leftarrow 0$ ;  $j < \epsilon_k$ ;  $j = j + 1$ )
3:      $b[\epsilon_k i + j] \leftarrow (f_i / 2^j) \bmod 2$ 
4:   end for
5: end for
6:  $B \leftarrow \text{BitsToBytes}(b)$ 
7: return  $B$ 
```

Algorithm 2 **BytesToPoly** $_{\epsilon_k}(B)$

Converts a byte array to an array of ϵ_k -bit integers.

Input: byte array $B \in \mathbb{B}^{\epsilon_k n/8}$

Output: polynomial coefficients $\{f_0, \dots, f_{n-1}\} \in \mathbb{Z}_{2^{\epsilon_k}}$

```

1:  $b \leftarrow \text{BytesToBits}(B)$ 
2: for ( $i \leftarrow 0$ ;  $i < n$ ;  $i = i + 1$ )
3:    $f_i \leftarrow 0$ 
4:   for ( $j \leftarrow 0$ ;  $j < \epsilon_k$ ;  $j = j + 1$ )
5:      $f_i \leftarrow f_i + b[\epsilon_k i + j] \cdot 2^j \bmod 2^{\epsilon_k}$ 
6:   end for
7: end for
8: return  $f$ 
```

Sampling from a Centered Binomial Distribution If s is sampled from a set S (e.g., $R_{q,n}$) according to a centered binomial distribution (CBD) β_η , this is denoted as $s \leftarrow \beta_\eta(S)$ and parametrized by η . For all security levels of Scabbard, the secret (vectors of) polynomials are sampled from β_η with fixed $\eta = 2$.

Sampling a secret polynomial from $\beta_\eta(R_{q,n})$ in Scabbard is achieved by independently sampling each of its coefficients by computing $\sum_{i=0}^{\eta-1} (a_i - b_i)$ in which a_i, b_i are uniformly random from $\{0, 1\}^\eta$. This functionality is described in **SampleSecretCBD** (Algorithm 3), tailored for $\eta = 2$, meaning each coefficient lies in $\{-2, -1, \dots, 2\}$.

Algorithm 3 `SampleSecretCBD`(σ)

Outputs ℓ pseudorandom polynomials from the distribution $\beta_2(R_{q,n})$, derived from an input seed.

Input: byte array $\sigma \in \mathbb{B}^\kappa$

Output: vector of polynomials $\mathbf{s} \in R_{q,n}^\ell$

```
1: for ( $i \leftarrow 0$ ;  $i < \ell$ ;  $i = i + 1$ )
2:    $B \leftarrow \text{PRF1}_{\kappa,n}(\sigma \parallel i)$   $\triangleright$  byte array  $B \in \mathbb{B}^{n/2}$ 
3:    $b \leftarrow \text{BytesToBits}(B)$ 
4:   for ( $j \leftarrow 0$ ;  $j < n$ ;  $j = j + 1$ )
5:      $x \leftarrow b[4j] + b[4j + 1]$ 
6:      $y \leftarrow b[4j + 2] + b[4j + 3]$ 
7:      $\mathbf{s}[i][j] \leftarrow x - y \bmod q$   $\triangleright \mathbf{s}[i][j] \in \mathbb{Z}_q$ 
8:   end for
9: end for
10: return  $\mathbf{s}$ 
```

While each secret polynomial's coefficient could be encoded in 3 bits, in *Scabbard* they are encoded within 4 bits using `PolyToBytes`₄, resulting in efficient (fast) packing of the polynomials into a bytestream (and vice versa). When unpacking the secret coefficients from a byte stream to their polynomial form (array of integers), each 4-bit chunk is interpreted as a signed integer in two's complement representation.

Sampling from a Uniform Distribution In *Scabbard*, the entries in the public matrix $\mathbf{A}$ are sampled from a uniform distribution on $R_{q,n}$, derived from a uniformly random seed. As described in `SampleMatrixUD` (Algorithm 4), each coefficient is sampled independently from a pseudorandom stream of bytes and will be in $\mathbb{Z}_q$.

Algorithm 4 `SampleMatrixUD`(ρ)

Outputs a matrix containing $\ell \times \ell$ pseudorandom entries, derived from a seed.

Input: byte array $\rho \in \mathbb{B}^\kappa$

Output: matrix of polynomials $\mathbf{A} \in R_{q,n}^{\ell \times \ell}$

```
1: for ( $i \leftarrow 0$ ;  $i < \ell$ ;  $i = i + 1$ )
2:   for ( $j \leftarrow 0$ ;  $j < \ell$ ;  $j = j + 1$ )
3:      $B \leftarrow \text{PRF2}_{\kappa,n,\epsilon_q}(\rho \parallel i \parallel j)$   $\triangleright$  byte array  $B \in \mathbb{B}^{n\epsilon_q/8}$ 
4:      $b \leftarrow \text{BytesToBits}(B)$ 
5:     for ( $k \leftarrow 0$ ;  $k < n$ ;  $k = k + 1$ )
6:        $\mathbf{A}[i][j][k] \leftarrow b[\epsilon_q k : \epsilon_q k + \epsilon_q]$   $\triangleright \mathbf{A}[i][j][k] \in \mathbb{Z}_q$ 
7:     end for
8:   end for
9: end for
10: return  $\mathbf{A}$ 
```

Message Encoding and Decoding For encoding the message part in the ciphertext, *Scabbard* uses Minal codes. To improve the correctness probability, an extension of 2D Minal codes [30] for error correction is defined that encodes 4 bits into two coefficients in $\mathbb{Z}_q$. Minal codes [30] are d -dimensional error correction codes with coefficients in $\mathbb{Z}_q$. Although inspired by lattice codes, they differ in one critical aspect: Minal codes are periodic in $\mathbb{Z}_q^d$, which makes them useful for lattice-based schemes where operations are typically done modulo q . In the case of *Scabbard*, 2D Minal codes are particularly interesting as they admit efficient decoders. For a positive integer q , and a non-negative integer $\beta < q/2$, the 2D Minal code with parameter β is the set $\mathcal{C} = \{\mathbf{G}\mathbf{m} + q\mathbf{z} : \mathbf{z} \in \mathbb{Z}^d \text{ and } \mathbf{m} \in \mathbb{Z}_2^2\}$, where $\mathbf{G} \in \mathbb{Z}^{2 \times 2}$ is the circulant matrix generated by $[\lfloor q/2 \rfloor, \beta]$. Matrix $\mathbf{G}$ is called the generator matrix of $\mathcal{C}$ and β is called the tailoring parameter. We say that $\mathbf{c}$ encodes a 2-bit message $\mathbf{m} \in \mathbb{Z}_2^2$ when $\mathbf{c} = \mathbf{G}\mathbf{m} + q\mathbf{z}$, for some $\mathbf{z} \in \mathbb{Z}^d$.

Since 2D Minal codes encode 2 bits into 2 coefficients, they can only be used directly in KEMs for which the message size μ is exactly equal to the degree n , used to define the ring $R_q = \mathbb{Z}_q[x]/(x^n + 1)$. However, in *Scabbard*, $\mu = 2n$ is used. To enable the smaller polynomial degrees, an extension of Minal

codes (“B2 Minal”) is defined, enabling the encoding of 4 bits into 2 coefficients in $\mathbb{Z}_q$. First, let $\mathbf{G}_1$ be the generator matrix of the 2D pure Minal code in $\mathbb{Z}_{\lfloor q/2 \rfloor}$. We then build a generator matrix $\mathbf{G}_2$ of the B2-2D-Minal code as

$$\mathbf{G}_2 = \left[\begin{array}{cc|c} \lfloor q/2 \rfloor & 0 & \mathbf{G}_1 \\ 0 & \lfloor q/2 \rfloor & \end{array} \right] = \left[\begin{array}{cc|cc} \lfloor q/2 \rfloor & 0 & \lfloor q/4 \rfloor & \beta \\ 0 & \lfloor q/2 \rfloor & \beta & \lfloor q/4 \rfloor \end{array} \right]. \quad (1.7)$$

MinalB2.Encode (Algorithm 5) shows how the encoding operation of a 4-bit message $\mathbf{m}$ corresponds to the computation of $\mathbf{c} = \mathbf{G}_2 \mathbf{m} \in \mathbb{Z}_q^2$. The decoding of a noisy codeword, shown in **MinalB2.Decode** (Algorithm 6), is then done in two phases. First we use a regular 2D Minal decoder to obtain the last two bits (m_2, m_3) of the message, as these were encoded based on the 2D Minal code generated by $\mathbf{G}_1$. Then, in the second phase, we obtain the first two bits (m_0, m_1) of the message by checking which of the 4 possibilities for (m_0, m_1) yields the codeword $\mathbf{G}_2[m_0, m_1, m_2, m_3]^\top$ closest to the target noisy codeword $\mathbf{c}$.

Algorithm 5 **MinalB2.Encode**($\mathbf{m}$)

Input: Message bits $\mathbf{m} \in \mathbb{Z}_2^4$

Output: Codeword $\mathbf{c} \in \mathbb{Z}_q^2$

- 1: $\mathbf{c} \leftarrow \mathbf{G}_2 \mathbf{m} \in \mathbb{Z}_q^2$ $\triangleright \mathbf{G}_2$ given in Eq. (1.7)
 - 2: **return** $\mathbf{c}$
-

Algorithm 6 **MinalB2.Decode**($\mathbf{c}$)

Input: Noisy codeword $\mathbf{c} \in \mathbb{Z}_q^2$

Output: Message bits $\mathbf{m} \in \mathbb{Z}_2^4$

Phase 1: Decode bits (m_2, m_3)

- 1: $\mathbf{c}' \leftarrow \mathbf{c} \bmod \lfloor q/2 \rfloor$ $\triangleright \mathbf{c}'$ is a regular Minal codeword in $\mathbb{Z}_{\lfloor q/2 \rfloor}^2$
- 2: $\text{min_dist} \leftarrow \infty$
- 3: **for** $\mathbf{v}$ in $\{(0, 0), (0, 1), (1, 0), (1, 1)\}$ **do** $\triangleright$ Minimum distance decoding for pure Minal
- 4: $d \leftarrow \|\mathbf{c}' - \mathbf{G}_1 \mathbf{v} \bmod \pm \lfloor q/2 \rfloor\|_2$
- 5: **if** $d < \text{min_dist}$ **then**
- 6: $\text{min_dist} \leftarrow d$
- 7: $(m_2, m_3) \leftarrow \mathbf{v}$
- 8: **end if**
- 9: **end for**

Phase 2: Decode bits (m_0, m_1)

- 10: $\mathbf{c}_{\text{best}} \leftarrow \mathbf{G}_1 [m_2 \ m_3]^\top$
 - 11: $(\delta_x, \delta_y) \leftarrow \mathbf{c} - \mathbf{c}_{\text{best}} \bmod \pm q$
 - 12: $m_0 \leftarrow \lfloor |\delta_x| / \lfloor q/4 \rfloor \rfloor$
 - 13: $m_1 \leftarrow \lfloor |\delta_y| / \lfloor q/4 \rfloor \rfloor$
 - 14: **return** $\mathbf{m} = (m_0, m_1, m_2, m_3)$
-

The full message encoding/decoding for Scabbard is described in **EncodeMessage** (Algorithm 7) and **DecodeMessage** (Algorithm 8).

Algorithm 7 **EncodeMessage**($\mathbf{m}$)

Encode a secret payload/message in polynomial form.

Input: Message bits $\mathbf{m} \in \mathbb{Z}_2^\mu$ (where $\mu = 2n$)

Output: Codeword $\mathbf{c} \in \mathbb{Z}_q^n$

- 1: $\mathbf{c} \leftarrow \mathbf{0} \in \mathbb{Z}_q^n$
 - 2: **for** $i = 0$ to $\mu/4$ **do**
 - 3: $\mathbf{c}[i], \mathbf{c}[i + n/2] \leftarrow \text{MinalB2.Encode}(\mathbf{m}[4i], \mathbf{m}[4i + 1], \mathbf{m}[4i + 2], \mathbf{m}[4i + 3]) \in \mathbb{Z}_q^2$
 - 4: **end for**
 - 5: **return** $\mathbf{c}$
-

Algorithm 8 `DecodeMessage(c)`

Decode a secret payload/message.

Input: Noisy codeword $\mathbf{c} \in \mathbb{Z}_q^n$

Output: Message bits $\mathbf{m} \in \mathbb{Z}_2^\mu$ (where $\mu = 2n$)

```
1:  $\mathbf{c} \leftarrow \mathbf{0} \in \mathbb{Z}_q^n$ 
2: for  $i = 0$  to  $\mu/4$  do
3:    $\mathbf{m}[4i], \mathbf{m}[4i+1], \mathbf{m}[4i+2], \mathbf{m}[4i+3] \leftarrow \text{MinalB2.Decode}(\mathbf{c}[i], \mathbf{c}[i+n/2]) \in \mathbb{Z}_q^4$ 
4: end for
5: return  $\mathbf{c}$ 
```

Polynomial Multiplication While selecting the moduli in Scabbard as power-of-two has several advantages (see Section 1.5), it also means the number theoretic transform (NTT) cannot be used for efficient polynomial multiplication. However, several other efficient polynomial multiplication algorithms exist, which offer similar performance for the chosen polynomial sizes (n coefficients). More specifically, Toom-Cook [14] and Karatsuba-based [21] polynomial multiplication are proposed for Scabbard, in which a larger polynomial multiplication is broken down into several smaller ones. Scabbard adopts 2 levels of Karatsuba (Toom-Cook 2-way) multiplication and 16×16 polynomial multiplication. However, this choice is not fixed and a different polynomial multiplication algorithm can be selected by the implementer, based on the target platform or optimization goal.

1.2 Specification of Scabbard.PKE

Scabbard is a MLWR-based scheme, in which noise is deterministically introduced by rounding from one number field (modulus q) to another one (modulus p), similar to Saber [16]. It is a variant of the learning with errors (LWE) problem [32] (and its module variant MLWE [10]), in which noise needs to be explicitly sampled from a distribution, which increases the computational cost of sampling and pseudorandomness generation.

The Scabbard PKE scheme consists of three main algorithms: `Scabbard.PKE.KeyGen` (Algorithm 9), `Scabbard.PKE.Encrypt` (Algorithm 10) and `Scabbard.PKE.Decrypt` (Algorithm 11).

Algorithm 9 `Scabbard.PKE.KeyGen()`

Generates a public encryption key and a corresponding secret decryption key.

Output: encryption key $\text{pk} \in \mathbb{B}^{\epsilon_p n \ell / 8 + \kappa}$

Output: decryption key $\text{sk} \in \mathbb{B}^{n \ell / 2}$

```
1:  $d \xleftarrow{\$} \mathbb{B}^\kappa$  ▷  $\kappa$  random bytes
2:  $\text{seed}_A \leftarrow \text{XOF}(d, \kappa)$  ▷ ensures RNG state is not revealed
3:  $\text{seed}_s \xleftarrow{\$} \mathbb{B}^\kappa$  ▷  $\kappa$  random bytes
4:  $\mathbf{A} \leftarrow \text{SampleMatrixUD}(\text{seed}_A)$  ▷  $\mathbf{A} \in R_{q,n}^{\ell \times \ell}$ 
5:  $\mathbf{s} \leftarrow \text{SampleSecretCBD}(\text{seed}_s)$  ▷ sample  $\mathbf{s} \in R_{q,n}^\ell$  from CBD
6:  $\mathbf{b} \leftarrow ((\mathbf{A}^\top \mathbf{s} + \mathbf{h}) \bmod q) \gg (\epsilon_q - \epsilon_p)$  ▷ coefficient-wise shift,  $\mathbf{b} \in R_{p,n}^\ell$ 
7:  $\text{pk} \leftarrow \text{PolyToBytes}_{\epsilon_p}(\mathbf{b}) \parallel \text{seed}_A$  ▷ call PolyToBytes  $\ell$  times
8:  $\text{sk} \leftarrow \text{PolyToBytes}_4(\mathbf{s})$  ▷ call PolyToBytes  $\ell$  times
9: return  $(\text{pk}, \text{sk})$ 
```

Algorithm 10 `Scabbard.PKE.Encrypt`(pk, m, r)

Generate a ciphertext using the encryption key, plaintext message and a random seed.

Input: encryption key $\text{pk} \in \mathbb{B}^{\epsilon_p n \ell / 8 + \kappa}$

Input: message $m \in \mathbb{B}^\kappa$

Input: randomness $r \in \mathbb{B}^\kappa$

Output: ciphertext $c \in \mathbb{B}^{(\epsilon_p \ell + \epsilon_t + 2)n/8}$

```
1:  $\mathbf{b} \leftarrow \text{BytesToPoly}_{\epsilon_p}(\text{pk}[0 : \epsilon_p n \ell / 8])$  ▷ call BytesToPoly  $\ell$  times
2:  $\text{seed}_A \leftarrow \text{pk}[\epsilon_p n \ell / 8 : \epsilon_p n \ell / 8 + \kappa]$ 
3:  $\mathbf{A} \leftarrow \text{SampleMatrixUD}(\text{seed}_A)$  ▷  $\mathbf{A} \in R_{q,n}^{\ell \times \ell}$ 
4:  $\mathbf{s}' \leftarrow \text{SampleSecretCBD}(r)$  ▷ sample  $\mathbf{s}' \in R_{q,n}^\ell$  from CBD
5:  $\mathbf{u} \leftarrow ((\mathbf{A}\mathbf{s}' + \mathbf{h}) \bmod q) \gg (\epsilon_q - \epsilon_p)$  ▷  $\mathbf{u} \in R_{p,n}^\ell$ 
6:  $v' \leftarrow \mathbf{b}^T \mathbf{s}' \bmod p$  ▷  $v' \in R_{p,n}$ 
7:  $\mu \leftarrow \text{EncodeMessage}(m)$  ▷  $\mu \in R_{2^2,n}$ 
8:  $v \leftarrow (2^{\epsilon_q - \epsilon_p} v' - \mu \bmod q) \gg (\epsilon_q - \epsilon_t - 2)$  ▷  $v \in R_{2^{(\epsilon_t + 2)},n}$ 
9:  $c_1 \leftarrow \text{PolyToBytes}_{\epsilon_p}(\mathbf{u})$  ▷ call PolyToBytes  $\ell$  times
10:  $c_2 \leftarrow \text{PolyToBytes}_{\epsilon_t + 2}(v)$ 
11: return  $c \leftarrow (c_1 \parallel c_2)$ 
```

Algorithm 11 `Scabbard.PKE.Decrypt`(sk, c)

Decrypts a ciphertext using the decryption key.

Input: decryption key $\text{sk} \in \mathbb{B}^{n \ell / 2}$

Input: ciphertext $c \in \mathbb{B}^{(\epsilon_p \ell + \epsilon_t + 2)n/8}$

Output: message $m \in \mathbb{B}^\kappa$

```
1:  $c_1 \leftarrow c[0 : \epsilon_p n \ell / 8]$ 
2:  $c_2 \leftarrow c[\epsilon_p n \ell / 8 : (\epsilon_p \ell + \epsilon_t + 2)n/8]$ 
3:  $\mathbf{u}' \leftarrow \text{BytesToPoly}_{\epsilon_p}(c_1)$  ▷ call BytesToPoly  $\ell$  times
4:  $v' \leftarrow \text{BytesToPoly}_{\epsilon_t + 2}(c_2)$ 
5:  $\mathbf{s}' \leftarrow \text{BytesToPoly}_4(\text{sk})$  ▷ call BytesToPoly  $\ell$  times
6:  $\mathbf{s} \leftarrow (\mathbf{s}' \oplus 8) - 8 \bmod p$  ▷ coefficient-wise, interpret as signed integer
7:  $w \leftarrow \mathbf{u}'^T \mathbf{s} \bmod p$  ▷  $w \in R_{p,n}$ 
8:  $m' \leftarrow (2^{\epsilon_q - \epsilon_p} w - 2^{\epsilon_q - \epsilon_t - 2} v') \bmod q$ 
9:  $m \leftarrow \text{DecodeMessage}(m')$ 
10: return  $m$ 
```

1.3 Specification of Scabbard.KEM

The IND-CCA2-secure Scabbard KEM is now constructed from the IND-CPA-secure PKE, described in Section 1.2, through a tweaked FO transform [17].

The Scabbard KEM scheme consists of three main algorithms: `Scabbard.KEM.KeyGen` (Algorithm 12), `Scabbard.KEM.Encaps` (Algorithm 13) and `Scabbard.KEM.Decaps` (Algorithm 14).

Algorithm 12 `Scabbard.KEM.KeyGen`()

Generates a public encapsulation key and a corresponding secret decapsulation key.

Output: encapsulation key $\overline{\text{pk}} \in \mathbb{B}^{\epsilon_p n \ell / 8 + \kappa}$

Output: decapsulation key $\overline{\text{sk}} \in \mathbb{B}^{(\epsilon_p + 4)n \ell / 8 + 3\kappa}$

```
1:  $(\text{pk}, \text{sk}) \leftarrow \text{Scabbard.PKE.KeyGen}()$ 
2:  $\overline{\text{pk}} \leftarrow \text{pk}$  ▷ encapsulation key is the encryption key
3:  $z \xleftarrow{\$} \mathbb{B}^\kappa$  ▷  $\kappa$  random bytes
4:  $\overline{\text{sk}} \leftarrow (\text{sk} \parallel \overline{\text{pk}} \parallel \text{XOF}(\text{pk}, \kappa) \parallel z)$  ▷ decapsulation key includes the decryption key
5: return  $(\overline{\text{pk}}, \overline{\text{sk}})$ 
```

Algorithm 13 `Scabbard.KEM.Encaps`($\overline{\text{pk}}$)

Generate a shared key and associated ciphertext using the encapsulation key.

Input: encapsulation key $\overline{\text{pk}} \in \mathbb{B}^{\epsilon_p n \ell / 8 + \kappa}$

Output: shared secret key $K \in \mathbb{B}^\kappa$

Output: ciphertext $c \in \mathbb{B}^{(\epsilon_p \ell + \epsilon_t + 2)n / 8}$

```
1:  $m \xleftarrow{\$} \mathbb{B}^\kappa$ 
2:  $m \leftarrow \text{XOF}(m, \kappa)$  ▷ ensures RNG state is not revealed
3:  $(K, r) \leftarrow \text{XOF}(m \parallel \text{XOF}(\overline{\text{pk}}, \kappa), 2\kappa)$ 
4:  $c \leftarrow \text{Scabbard.PKE.Encrypt}(\overline{\text{pk}}, m, r)$ 
5:  $K \leftarrow \text{XOF}(K, \text{XOF}(c, \kappa), \kappa)$ 
6: return  $(K, c)$ 
```

Algorithm 14 `Scabbard.KEM.Decaps`($\overline{\text{sk}}, c$)

Generate a shared key from a ciphertext using the decapsulation key.

Input: decapsulation key $\overline{\text{sk}} \in \mathbb{B}^{(\epsilon_p + 4)n \ell / 8 + 3\kappa}$

Input: ciphertext $c \in \mathbb{B}^{(\epsilon_p \ell + \epsilon_t + 2)n / 8}$

Output: shared secret key $K \in \mathbb{B}^\kappa$

```
1:  $\text{sk} \leftarrow \overline{\text{sk}}[0 : n \ell / 2]$ 
2:  $\text{pk} \leftarrow \overline{\text{sk}}[n \ell / 2 : (\epsilon_p + 4)n \ell / 8 + \kappa]$ 
3:  $\text{pkh} \leftarrow \overline{\text{sk}}[(\epsilon_p + 4)n \ell / 8 + \kappa : (\epsilon_p + 4)n \ell / 8 + 2\kappa]$ 
4:  $z \leftarrow \overline{\text{sk}}[(\epsilon_p + 4)n \ell / 8 + 2\kappa : (\epsilon_p + 4)n \ell / 8 + 3\kappa]$ 
5:  $m' \leftarrow \text{Scabbard.PKE.Decrypt}(\text{sk}, c)$ 
6:  $(K', r') \leftarrow \text{XOF}(m' \parallel \text{pkh}, 2\kappa)$ 
7:  $c' \leftarrow \text{Scabbard.PKE.Encrypt}(\text{pk}, m', r')$ 
8: if  $c = c'$  then
9:    $K \leftarrow \text{XOF}(K', \text{XOF}(c, \kappa), \kappa)$ 
10: else
11:    $K \leftarrow \text{XOF}(z, \text{XOF}(c, \kappa), \kappa)$ 
12: end if
13: return  $K$ 
```

1.4 Scabbard Parameter Sets

Three parameter sets are proposed for **Scabbard**, corresponding to classical security strengths of (at least) 128, 256 and 512 bits and quantum security strength of (at least) 80, 128, and 256 bits, referred to as: **Scabbard-128**, **Scabbard-256** and **Scabbard-512**, respectively. All selected parameters are shown in Table 1, alongside a description of their role and reasoning for why they were chosen. In addition, for all variants, the CBD parameter $\eta = 2$ is constant with resulting coefficients encoded in 4 bits, and 2 secret payload bits are encoded per coefficient ($B = 2$).

Table 1: Proposed parameter sets for **Scabbard**

	ℓ	n	q	p	t	κ	B2-2D-Minal β
Scabbard-128	9	64	2^{14}	2^{10}	2^3	16	1030
Scabbard-256	9	128	2^{13}	2^{11}	2^2	32	0
Scabbard-512	8	256	2^{13}	2^{11}	2^6	64	550

- n is chosen from $\{64, 128, 256\}$ as the goal of **Scabbard** is to establish a shared secret between two parties with $\{128, 256, 512\}$ bits of entropy. This requires encoding multiple key bits into one polynomial coefficient. Together with ℓ , it determines the lattice dimension $\ell \cdot n$, which determines the security.
- All moduli q, p, t are chosen as powers of two. This has several advantages: implicit modular reduction, and no need for complex and non-constant time rejection sampling routines. They are

selected to provide a balance between performance and achieving a negligible failure probability, required for CCA security.

- The CBD parameter η is chosen to balance security and failure probability. It is fixed across Scabbard instantiations, simplifying implementations.

1.5 Design Rationale

One of the primary design objectives of Scabbard is to achieve extreme parallelizability and a minimal memory footprint for resource-constrained embedded devices while maintaining high performance on larger platforms through module-space exploration. Scabbard adopts a similar approach as proposed in modern lattice-based schemes such as Rudraksh [24], by selecting a unique design point in the lattice parameter space, bridging the gap between theoretical security and practical resource constraints.

Module LWR The security of Scabbard is based on the hardness of the MLWR [7, 27] problem. By utilizing a deterministic rounding operation to implicitly generate errors, Scabbard eliminates the need for explicit error sampling as required in (Module-)LWE schemes like Kyber [10]. This design choice significantly reduces the requirement for pseudo-random numbers, leading to better performance and smaller public-key and ciphertext sizes for equivalent security levels. Furthermore, the lack of explicit error sampling provides natural protection against certain fault-injection and side-channel attacks that exploit seed manipulation during error vector generation [31].

Non-Fixed Polynomial Degree Scabbard adopts a tiered parameterization in which the security level is scaled by increasing the polynomial degree $n \in \{64, 128, 256\}$. It is contrary to other techniques, which solely rely on increasing the module rank l to scale to different security levels and fix $n = 256$, Scabbard’s strategy of gradually increasing n with security provides a unique trade-off in hardware parallelism and resource consumption. For example, a use-case where low resource consumption is more important than low latency, requiring lower levels of security: Scabbard offers the choice to use the lower degree polynomials, such as $n = 64$, which requires fewer hardware resources (such as FPGA DSP units) than a larger $n = 256$ unit. However, for a high security instantiation with high performance requirements, such as a hardware security module (HSM), low resource consumption is of lesser priority. Here, one can use the $n = 128$ or $n = 256$ parameter sets and instantiate many parallel multipliers. In this situation, the latency of one multiplier becomes important and thus other parameters, such as the variance of the central binomial distribution (CBD) ($\eta = 2$), become important. In Scabbard, all multiplications of two random elements contain one multiplicand which is small (due to the parameter choices), allowing multiplication to be implemented using simple shifts and additions/subtractions, in contrast to multiplications in the NTT domain. Finally, the inherent flexibility of Scabbard enables one to instantiate the low security versions in a parallel fashion, or the high security version in a serial manner to save memory and other resources, if desired.

Small Polynomial Ring Size A fundamental departure from established standards (which typically use a fixed degree of $n = 256$) is Scabbard’s use of different sizes of polynomials. Notably, the smallest polynomial degree in Scabbard is $n = 64$. This design choice, inspired by the need for fine-grained control over security and resources, offers two primary advantages:

- *Low Memory Footprint:* on software platforms like the ARM Cortex-M4, Scabbard can achieve a low stack memory footprint.
- *Hardware Parallelism:* the small polynomial size allows hardware designers to exploit high levels of parallelism, as a $n = 64$ multiplier requires significantly fewer DSP units (as few as 4 DSPs) compared to a $n = 256$ multiplier. As a result, multiple parallel multipliers can be instantiated on an FPGA to perform matrix-vector and vector-vector multiplications in batch without exhausting silicon area.

Power-of-Two Moduli To ensure robust side-channel attack (SCA) resilience, Scabbard utilizes power-of-two moduli (q, p, t in Table 1). This eliminates the need for explicit modular reduction and rejection sampling, which are especially expensive in the masked domain. Specifically, the choice of

power-of-two moduli in **Scabbard** facilitates efficient arbitrary-order masking by making arithmetically expensive operations such as arithmetic-to-Boolean (A2B) conversions significantly cheaper than required in prime-modulus schemes.

High-Density Secret Embedding Unlike traditional designs of lattice-based cryptographic schemes, where each coefficient of a polynomial carries 1 bit of shared secret *i.e.* $B = 1$, **Scabbard** carries 2 bits of secret ($B = 2$) in each polynomial coefficient. This decision allows the scheme to perfectly align the total secret entropy with the ring dimension; specifically, a 64-degree polynomial ($n = 64$) can carry the full 128-bit secret without redundancy. In contrast, designs limited to 1 bit per coefficient ($B = 1$) would require a polynomial degree of at least 128 or larger. By matching the shared secret density to the polynomial size, **Scabbard** significantly reduces the memory footprint and resource requirements, which are vital in resource-constrained IoT environments.

Polynomial Multiplication Polynomial multiplication is the primary computational bottleneck in lattice-based cryptography. Unlike prime-modulus schemes, which utilize the NTT to achieve a polynomial multiplication with $O(n \log n)$ complexity, **Scabbard** is designed with both the modulus q and the ring dimension n as powers of two. This algebraic structure precludes the *direct* use of standard NTT-based multiplication. To maintain high efficiency without a prime modulus, **Scabbard** adopts Toom-Cook and Karatsuba-based polynomial multiplication. As shown for Saber [16], for smaller degree polynomials, Toom-Cook and Karatsuba-based polynomial multiplications perform at par or even better than NTT-based multiplications despite being slower asymptotically. This mainly stems from the fact that for a power-of-two moduli, the modular reduction in Toom-Cook- and Karatsuba-based multiplications is *free* compared to the expensive Barrett or Montgomery modular reduction algorithms used in NTT.

2D Minal Encoding As for most lattice-based KEMs, the decryption algorithm of **Scabbard** needs to correct a noisy version of the encapsulated message. To achieve negligible values for the probability of observing decryption failures, we first conduct a parameter search for combinations that ensures theoretical security, and then, for selected parameters, we tailored the B2 Minal codes parameter β to minimize decryption failure probability. We show a briefly analysis of the decryption failure probability as follows.

Let $\mathbf{e}$ and $\mathbf{e}'$ be the rounding errors introduced by rounding $\mathbf{A}^\top \mathbf{s}$ (key generation) and $\mathbf{A}\mathbf{s}'$ (encryption), respectively. Then we can write

$$m' = \text{Encode}(\text{msg}) + \Delta m,$$

where the accumulated noise polynomial $\Delta m = \langle \mathbf{e}, \mathbf{s}' \rangle - \langle \mathbf{s}', \mathbf{e} \rangle$. Therefore, using the B2-Minal codes for error correction, we can apply the union bound to obtain an upper bound on the decryption failure probability (DFP) as

$$\begin{aligned} \text{DFP} &\leq \sum_{i=0}^{n/2-1} \Pr(\text{MinalB2.Decode}(m'[i, i + n/2]) = \text{msg}[4i, 4i + 1, 4i + 2, 4i + 3]) \\ &= \frac{n}{2} \Pr(\text{MinalB2.Decode}(m'[0, n/2]) = \text{msg}[0, 1, 2, 3]) \\ &= \frac{n}{2} \Pr(\text{MinalB2.Decode}(\text{MinalB2.Encode}(\text{msg}[0, 1, 2, 3]) + \Delta m[0, n/2]) = \text{msg}[0, 1, 2, 3]). \end{aligned}$$

The computation of the failure probability can then be done as follows. First we compute the joint distribution $\Pr(\Delta m[0, n/2])$ of the noise on coordinates 0 and $n/2$. Then, for each possible 4-tuples of message bits $\text{msg}[0, 1, 2, 3]$, we iterate over all possible values of the noise $\Delta m[0, n/2]$ and check if decoding fails or not, updating the DFP accordingly. The most expensive step is the computation of the joint distribution $\Pr(\Delta m[0, n/2])$, which can be done following the analysis of Minal codes applied to ML-KEM [30]. In particular, as done in the analysis of regular Minal codes, we need to use FFT-based convolutions with arbitrary precision to efficiently compute for the full computation of $\Delta m[0, n/2]$. Therefore, to facilitate the security analysis of **Scabbard**, all scripts for computing the decryption failure probability will be made publicly available.

CCA2 Security Scabbard has been designed to ensure robust protection against chosen-ciphertext attacks (IND-CCA2), a necessity for deployments where public keys are static and reused across many sessions. Without this level of security, an active adversary could potentially extract sensitive information through multiple, carefully crafted ciphertext queries. To achieve this, Scabbard utilizes a modified FO transform to bridge the gap between its basic IND-CPA-secure public-key encryption and a fully secure KEM. Fundamentally, the security of this construction is ensured through a consistency check during decapsulation: the system re-encrypts the recovered plaintext internally and only authorizes the release of the shared secret if the generated candidate ciphertext is bit-for-bit identical to the one received. In the event of a mismatch between these two values or a rare decryption error, the mechanism avoids triggering explicit error flags, which could be exploited as leakage of information, and instead returns a deterministic, pseudo-random value. This value is derived from a unique seed (z) embedded within the recipient’s private key. This effectively neutralizes an attacker’s ability to gain information from malformed inputs or to launch adaptive attacks based on system responses.

1.6 Features

In summary, Scabbard is designed to achieve the following properties:

- **Simplicity:** the Scabbard construction is deliberately simple, relying on the well-understood LWR hard problem, and requiring relatively few function calls to implement due to its power-of-two moduli and modular structure.
- **Flexibility:** the polynomial multiplication, the primary bottleneck in lattice-based arithmetic, can be implemented using a variety of techniques and algorithms as it is not fixed in Scabbard. Depending on the target platform and optimization strategy, an optimal multiplication strategy can be chosen.
- **Performance Advantage:** the small polynomial ring size in Scabbard results in a low memory footprint on embedded devices or high levels of parallelism in hardware. Additionally, the choice of power-of-two moduli results in efficient side-channel protected implementations as no rejection sampling is required and (expensive) masked multiplication is not required.
- **Innovativeness:** Scabbard is designed with efficiency on both software and hardware platforms kept in mind, alongside security. The MLWR construction enables high parallelism, fine-grained parameter selection and reduced memory/area footprint on various target platforms. The use of extended 2D Minal encoding reduces the failure probability to a negligible range.
- **Compatibility:** both Scabbard and ATLAS are based on the MLWR hard problem and form a suite of next-generation post-quantum cryptographic schemes: MORNING (MOdule learning with RouNdiNG). Both have a similar structure and are designed with similar design goals.

2 Performance Evaluation

In this section, the implementation aspects of **Scabbard** are discussed and the performance of its ISO C reference implementation is analyzed. We remark that the optimized implementation included in the submission package, as requested by ICCS, is a copy of the reference implementation. Additionally, all function calls to symmetric primitives are implemented using the `pseudoHash` and `pseudoXOF` primitives, provided by ICCS.

2.1 Analysis, Considerations and Trade-offs

Scabbard is designed with efficiency and compactness on both software and hardware platforms beared in mind. We achieve relatively low computational complexity, compared to other LWE-based schemes, by design choices of a power-of-two moduli, and rounding via bit shifts, which directly affect the performance characteristics.

Power-of-two Moduli The moduli q , p , and t are all powers of two. This choice has two important performance consequences. It first eliminates rejection sampling during matrix generation. Since q is a power-of-two moduli, every ϵ_q bits value drawn from the hash output is a valid element of $\mathbb{Z}_q$, so no output bit needs to be discarded and no resampling is required, ensuring a constant-time and deterministic implementation and no performance drop. Second, the choice of power-of-two allows the rounding operation to be implemented as simple as a bitwise mask or bitwise AND in both software and hardware platforms. Moreover, the change of moduli can also be implemented efficiently as three cheap instructions: one addition, one bitwise AND, and one right shift.

Polynomial Multiplication The choice of the polynomial multiplication algorithm is not fixed and can be adjusted based on the target platform or optimization goal. In the reference implementation, Toom-Cook 2-way multiplication with 16×16 polynomial multiplication is used. However, we expect that both target platform-specific optimizations and exploration of other multiplication techniques can further improve performance.

2.2 Transmission and Storage Cost

The ciphertext and key sizes for all **Scabbard** parameter sets are summarized in Table 2.

Table 2: Key and ciphertext sizes (in bytes) for all **Scabbard** parameter sets

	encapsulation key	decapsulation key	ciphertext	shared secret key
Scabbard-128	736	1056	760	16
Scabbard-256	1616	2256	1648	32
Scabbard-512	2880	4032	3072	64

2.3 Performance of Scabbard on Intel CPUs

Both the performance results given below (Table 3 and Table 4) are obtained from running the reference implementation and `avx` implementation respectively of **Scabbard** on 13th Gen Intel Core i7. The machine has 32 GBs of RAM and is running Ubuntu 22.04.5. The implementations are compiled with gcc version 11.4.0 and compiler flags present in the Makefiles of the submission package. All reported cycle counts are the median of 10K executions.

2.3.1 Reference implementation

To do the benchmarking of reference implementation, we have used “-std=c99 -Wpedantic -Wall -Wextra -O2” as flags.

Table 3: Cycle counts of the C reference implementation for all parameter sets of Scabbard, obtained on one core of an 13th Gen Intel Core i7.

	Key Generation	Encapsulation	Decapsulation
Scabbard-128	531,247	557,866	564,095
Scabbard-256	1,271,337	1,366,187	1,414,029
Scabbard-512	3,783,915	4,149,324	4 317,833

2.3.2 AVX2 implementation

In the avx implementation we have used the flags “-std=c99 -Wpedantic -Wall -Wextra -D__AVX2 -mavx2 -O3 -march=x86-64 -mtune=native -fomit-frame-pointer” and get the benchmarking results in Table 4.

Table 4: Cycle counts of the AVX2 reference implementation for all parameter sets of Scabbard, obtained on one core of 13th Gen Intel Core i7.

	Key Generation	Encapsulation	Decapsulation
Scabbard-128	419,661	434,564	425,610
Scabbard-256	742,910	787,355	787,355
Scabbard-512	2,051,602	2,182,188	2,093,661

We integrated AVX2 optimizations into the `SampleSecretCBD`, polynomial multiplication, and rounding operations utilized across the `Scabbard.PKE.KeyGen` (Algorithm 9), `Scabbard.PKE.Encrypt` (Algorithm 10), and `Scabbard.PKE.Decrypt` (Algorithm 11) algorithms.

The AVX2 implementation parallelizes this exact process by utilizing 256-bit SIMD registers to process 32 bytes of randomness simultaneously, producing 64 coefficients at once. Instead of a scalar loop, it uses bitwise masks (mask55 and mask33) and vector shifts across all 256 bits at once. It calculates the Hamming weights of 4-bit chunks in parallel using byte-level additions and shifts. The subtraction of the binomial components ($a - b$) is done simultaneously across the vector lanes. It uses constant offsets (mask03 and mask0F) to cleanly isolate the bit fields and perform the subtraction in parallel on 8-bit element blocks. This expands the data smoothly from 8-bit chunks into four separate 256-bit registers which are then stored directly to memory. The AVX2 implementation parallelizes the rounding operations, processing 16 coefficients simultaneously during the rounding steps in Algorithm 9 (line 6), Algorithm 10 (line 5 and 8), and Algorithm 11 (line 8).

2.4 Performance of Scabbard on ARM Cortex-M4 MCUs

Table 5: Cycle counts of the optimized implementations for all parameter sets of Scabbard on cortex-m4.

	Key Generation	Encapsulation	Decapsulation
Scabbard-128	2,399,842	2,500,069	2,482,683
Scabbard-256	4,635,377	4,915,352	4,932,125
Scabbard-512	12,522,155	13,445,744	13,364,597

The reference design is ported to ARM Cortex-M4 platform. Some optimizations are implemented targeting the embedded execution environment. The performance results given in Table 5 are obtained from running the optimized implementations of Scabbard on STM32F407G-DISC1 development board with 24 Mhz. The implementations are cross-compiled on Ubuntu20.04.2 with arm-none-eabi-gcc version 9.2.1 (-o3 optimization flag) and compiler flags present in the Makefiles of the submission package. These optimized implementations on cortex-m4 are all constant-time implementations, meaning that there is no secret-dependent branches, jumps or cycle-variate instructions in the implementations.

Therefore the implementations are resistant against time side-channel attacks. Similarly to the reference C implementation in Section 2.3, around 70% of the clock cycles in the current implementations are also spent in the `pseudoXOF` function.

As mentioned in Section 2.1, the choice of polynomial multiplication algorithm is platform and optimization specific. We opt for schoolbook multiplication for the polynomial multiplication in all three configurations. The optimized ARM assembly code for these polynomial multiplications are generated using `polymul-z2mx-m4` library [20].

Table 6: Peak stack usage of the optimized implementations for all parameter sets of **Scabbard** on cortex-m4.

	Key Generation	Encapsulation	Decapsulation
Scabbard-128	1,992	2,624	2,656
Scabbard-256	2,712	3,952	3,984
Scabbard-512	5,260	7,588	7,620

To reduce stack usage on Cortex-M4, the implementation was redesigned to avoid keeping large intermediate objects on the stack. Instead of materializing full matrices, vectors, or temporary ciphertexts, the computation is performed in a streaming fashion.

In particular, the public matrix is generated on-the-fly from its seed, and intermediate polynomial-vector results are not stored as full objects. Secret data is kept in a compact representation where possible and expanded only locally when required. Similarly, during decapsulation, the re-encryption check compares the recomputed ciphertext with the received ciphertext on-the-fly, rather than first storing a complete recomputed ciphertext buffer.

This design trades a small amount of recomputation and local buffering for significantly lower stack usage, which is more suitable for constrained embedded targets such as Cortex-M4. Table 6 summarizes the stack usage for all implementations on Cortex-M4.

3 Security Analysis

Scabbard is an IND-CCA2-secure key encapsulation mechanism. Its security and correctness are summarized in Table 7.

Table 7: Summary of security and correctness of Scabbard.

	CSS	QSS	δ	Gates	Memory(bits)
Scabbard-128	2^{128}	2^{116}	2^{-99}	2^{161}	2^{100}
Scabbard-256	2^{266}	2^{241}	2^{-125}	2^{300}	2^{198}
Scabbard-512	2^{514}	2^{466}	2^{-125}	2^{534}	2^{365}

- The classical security strength (CSS) is defined as the Classical Core-SVP strength, and exceeds $\{128, 256, 512\}$ for the three Scabbard instantiations.
- The quantum security strength (QSS) is defined as the Quantum Core-SVP strength, and exceeds $\{80, 128, 256\}$ for the three Scabbard instantiations.
- The correctness of Scabbard is defined through the failure probability δ , which is sufficiently small.
- The estimated gate cost of classical attacks based on [2] defined as the Gates, and exceeds $\{128, 256, 512\}$ for the three Scabbard instantiations.
- The estimated bit memory requirements of classical attacks based on [2] defined as the Memory, and presented for the three Scabbard instantiations.

3.1 Theoretical Security

In this section, the correctness and security of the Scabbard.PKE and Scabbard.KEM protocols is discussed. According to this direction, we first explain the Scabbard key exchange protocol (Scabbard.KE, Figure 1) and its correctness and security. After that, we will provide a reduction from this Scabbard.KE to Scabbard.PKE and Scabbard.KEM. Here some of the functions `XOF`, `SampleMatrixUD` and `SampleMatrixUD` are defined before. The remaining functions are defined by:

`Encode`(x) = “most significant B bits of x ”

`HelpDecode`(x) = “next ϵ_t many bits of x ”

`Decode`(y , `HelpDecode`(x)) = $y - \text{HelpDecode}(x) \frac{q}{2^{B+\epsilon_t}}$

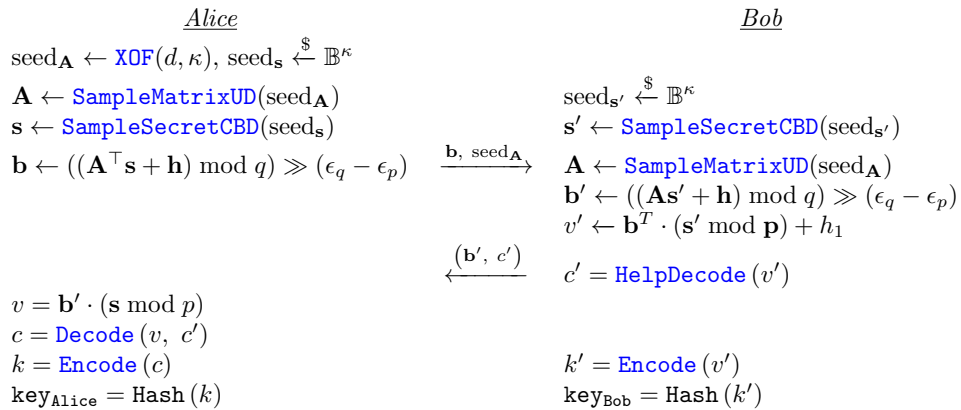


Figure 1: Scabbard.KE

In Scabbard.KE (Figure 1), two parties, Alice and Bob, will get the same shared key, i.e., $\text{key}_{\text{Alice}} = \text{key}_{\text{Bob}}$, if $k = k'$. Therefore the protocol will be correct if the following equality holds

$$\text{Encode}(v') = \text{Encode}(\text{Decode}(v, \text{HelpDecode}(v'))).$$

Suppose $v' - v = e$. The following theorem states that if the absolute error e is bounded by $\frac{q}{2^{B+1}} - \frac{q}{2^{B+\epsilon_t+1}}$, then the equality holds, i.e., the protocol will be correct. For the parameter set of Scabbard.KE, the absolute error will be bounded by $\frac{q}{2^{B+1}} - \frac{q}{2^{B+\epsilon_t+1}}$ with high probability. Therefore, this protocol will be correct with high probability.

Theorem 1. $\text{Encode}(x) = \text{Encode}(\text{Decode}(y, \text{HelpDecode}(x)))$ holds if $x = y + e$ and $|e| \leq \frac{q}{2^{B+1}} - \frac{q}{2^{B+\epsilon_t+1}}$ holds.

We say the protocol Scabbard.KE is $(1 - \delta)$ -correct if, the probability that the derived key k' matches the expected shared key k exceeds $(1 - \delta)$. The protocol Scabbard.KE is considered IND-RND secure if it is computationally infeasible for an adversary $\mathcal{A}$ to differentiate the actual shared secret k from a value chosen uniformly at random $\hat{k} \leftarrow \mathcal{U}(K)$, where K is the key space. i.e.,

$$\text{Adv}_{\text{Scabbard.KE}}^{\text{ind-rnd}}(\mathcal{A}) = \left| \Pr[\mathcal{A}(\mathbf{A}, \mathbf{b}, \mathbf{b}', c', k) = 1] - \Pr[\mathcal{A}(\mathbf{A}, \mathbf{b}, \mathbf{b}', c', \hat{k}) = 1] \right|$$

Now, we will define more security definitions as follows which are required to explain the security of Scabbard.KE, Scabbard.PKE and Scabbard.KEM.

$$\text{Adv}_{\text{gen}()}^{\text{prg}}(\mathcal{A}) = \left| \Pr \left[\begin{array}{l} r \leftarrow \mathcal{U}(\{0, 1\}^\kappa) \\ b' = 1 : s \leftarrow \text{gen}(r) \in (\{0, 1\}^n) \\ b' = \mathcal{A}(s) \end{array} \right] - \Pr \left(b' = 1 : \begin{array}{l} s \leftarrow \mathcal{U}(\{0, 1\}^n) \\ b' = \mathcal{A}(s); \end{array} \right) \right|$$

$$\text{Adv}_{m, \ell, \mu, q, p}^{\text{MLWR}}(\mathcal{A}) = \left| \Pr \left(b' = 1 : \begin{array}{l} \mathbf{A} \leftarrow \mathcal{U}(R_q^{m \times \ell}); \mathbf{s} \leftarrow \beta_\mu(R_q^{\ell \times 1}); \\ b' = A(\mathbf{A}, \lfloor (p/q)\mathbf{A}\mathbf{s} \rfloor); \end{array} \right) - \Pr \left(b' = 1 : \begin{array}{l} \mathbf{A} \leftarrow \mathcal{U}(R_q^{m \times \ell}); \mathbf{u} \leftarrow \mathcal{U}(R_p^{\ell \times 1}); \\ b' = A(\mathbf{A}, \mathbf{u}); \end{array} \right) \right|$$

In our scheme the underlying hard problem is based on Module-LWR. We define the following security games:

Definition 3.1. The advantage $\text{Adv}_{\text{Scabbard.CPAPKE}}^{\text{cpa}}$ is defined based on a "game" between a Challenger and an Adversary $(\mathcal{A})$:

- The Challenger runs **Scabbard.PKE.KeyGen** to generate public key and secret key pair (pk, sk) .
- The Adversary chooses two messages m_0 and m_1 of equal length and sends them to the Challenger.
- The Challenger choose a bit b randomly and computes the ciphertext $(\mathbf{u}, v) = \text{Scabbard.PKE.Encrypt}(\text{pk}, m_b)$, and sends $(\mathbf{u}, v)$ to the Adversary.
- The Adversary $\mathcal{A}$ outputs a guess b' .

The advantage is given by

$$\text{Adv}_{\text{Scabbard.CPAPKE}}^{\text{cpa}}(\mathcal{A}) = \left| \Pr[b = b'] - \frac{1}{2} \right|$$

Definition 3.2. The advantage $\text{Adv}_{\text{Scabbard.KEM}}^{\text{cca}}$ is defined based on a "game" between a Challenger and an Adversary $(\mathcal{A})$:

- The Challenger runs **Scabbard.KEM.KeyGen** to generate public key and secret key pair $(\overline{\text{pk}}, \overline{\text{sk}})$.
- The adversary $\mathcal{A}$ send any ciphertext $c = (\mathbf{u}, v)$ to the Challenger. The Challenger runs **Scabbard.KEM.Decaps** with the input $(c, \overline{\text{sk}})$ and returns the session key K as outputs. The adversary can do this query for multiple ciphertexts.
- The Challenger choose a bit b randomly and compute $K = \begin{cases} \text{Scabbard.KEM.Encaps}(\text{pk}) & \text{if } b = 0 \\ \text{random key} & \text{if } b = 1 \end{cases}$. Send this key K to the adversary.

- The Adversary can query the decapsulation or encapsulation oracles with any ciphertext except this key K and then outputs a guess b' .

The advantage is given by

$$Adv_{Scabbard.KEM}^{cca}(\mathcal{A}) = \left| \Pr[b = b'] - \frac{1}{2} \right|.$$

Theorem 2. *MLWR based Scabbard.KE is IND-RND secure if $\frac{q}{p} \leq \frac{p}{2^{B_t}}$.*

Proof. The proof of this theorem requires to show that for any adversary $\mathcal{A}$, the advantage $Adv_{Scabbard.KE}^{ind-rnd}(\mathcal{A})$ is negligible for $\frac{q}{p} \leq \frac{p}{2^{B_t}}$. This result follows directly from the lemma below. $\square$

Lemma 3. *For any adversary $\mathcal{A}$, there exist three adversaries $\mathcal{B}_0$, $\mathcal{B}_1$ and $\mathcal{B}_2$ such that $Adv_{Scabbard.KE}^{ind-rnd}(\mathcal{A}) \leq Adv_{gen()}^{prg}(\mathcal{B}_0) + Adv_{\ell,\ell,\mu,q,p}^{MLWR}(\mathcal{B}_1) + Adv_{\ell+1,\ell,\mu,q,p}^{MLWR}(\mathcal{B}_2)$, if $\frac{q}{p} \leq \frac{p}{2^{B_t}}$.*

Proof. The proof follows the paper [9]. $\square$

3.1.1 Correctness

Theorem 4. *For every adversary $\mathcal{A}$ targeting the IND-CPA security of Scabbard.PKE, there exists an adversary $\mathcal{B}$ targeting the IND-RND security of Scabbard.KE such that their advantages are equivalent: $Adv_{Scabbard.PKE}^{ind-cpa}(\mathcal{A}) = Adv_{Scabbard.KE}^{ind-rnd}(\mathcal{B})$. Additionally, the correctness of Scabbard.PKE and Scabbard.KE are identical; specifically, Scabbard.PKE is $(1 - \delta)$ -correct if and only if Scabbard.KE is $(1 - \delta)$ -correct.*

Following Theorem 1, the correctness of the Scabbard.PKE protocol can be evaluated via the Python scripts included in the submission package.

3.1.2 IND-CCA security proof of Scabbard in the ROM

To prove an upper bound on CCA security, we assume the XOF function work as random oracles. As we use a KEM variant of the Fujisaki-Okamoto (FO) transformation, we use a similar security bound from the paper of Hofheinz et al. [18]. Since the failure probability of Scabbard.PKE is δ , the failure probability of Scabbard.KEM will be the same. To ensure the security proofs remain valid, this failure probability δ must be cryptographically negligible. Consequently, we get the following theorem regarding the security of our KEM within the classical random oracle model by applying Theorem 3.2 and Theorem 3.4 from the work of Hofheinz et al [18].

Theorem 5. [18] *For any IND-CCA adversary $\mathcal{B}$ that makes at most q_{RO} queries to the random oracle XOF, respectively, and q_D queries to the decryption oracle, there exists an IND-CPA adversary $\mathcal{A}$ such that the advantage is bounded as follows:*

$$Adv_{Scabbard.KEM}^{ind-cca}(\mathcal{B}) \leq 3Adv_{Scabbard.PKE}^{ind-cpa}(\mathcal{A}) + q_{RO} \cdot \delta + \frac{3q_{RO}}{2^{8\kappa}}$$

3.1.3 IND-CCA security proof of Scabbard in the QROM

In paper [19], Jiang et al. provide the reduction of IND-CCA and IND-CPA securities against a quantum adversary using quantum oracles. Following this, we can provide a reduction from IND-CCA to IND-CPA security for Scabbard.

Theorem 6. [19] *Given a quantum IND-CCA adversary $\mathcal{B}$ that makes at most q_{RO} queries to XOF, q_D decryption queries, and message space is $\mathcal{M}$ we can construct an adversary $\mathcal{A}$ such that:*

$$Adv_{Scabbard.KEM}^{ind-cca}(\mathcal{B}) \leq 2q_{RO} \frac{1}{\sqrt{|\mathcal{M}|}} + 4q_{RO}\sqrt{\delta} + 4q_{RO}\sqrt{Adv_{Scabbard.PKE}^{ind-cpa}(\mathcal{A}) + \frac{1}{|\mathcal{M}|}}$$

3.2 Practical Security

3.2.1 Attacks against the underlying MLWR problem

In our scheme, the underlying hard problem is based on Module-LWR (MLWR). As there is no known attack on MLWR structures, we treat the hardness of MLWR as an instance of the LWE problem. We transform l many LWR instances to LWE instances in the following ways:

$$(\mathbf{A}, \mathbf{b}) = \left(\mathbf{A}, \left\lfloor \frac{p}{q} (\mathbf{A}^\top \mathbf{s} \bmod q) \right\rfloor \bmod p \right) = \left(\mathbf{A}, \frac{p}{q} (\mathbf{A}^\top \mathbf{s} \bmod q) + \mathbf{e} \bmod p \right)$$

where $\mathbf{A} \leftarrow \mathcal{U}(R_{q,n}^{\ell \times \ell})$, $\mathbf{s} \leftarrow \beta_\mu(R_{q,n}^{\ell \times 1})$, $\mathbf{e}$ is the rounding error, i.e., $\mathbf{e} = \left\lfloor \frac{p}{q} (\mathbf{A}^\top \mathbf{s} \bmod q) \right\rfloor - \frac{p}{q} (\mathbf{A}^\top \mathbf{s} \bmod q) \bmod p$. Here, we can write $\mathbf{b}$ as $\mathbf{b} = \frac{p}{q} (\mathbf{A}^\top \mathbf{s} \bmod q) + \mathbf{e} \bmod p$, i.e., $\frac{q}{p} \mathbf{b} = \mathbf{A}^\top \mathbf{s} + \frac{q}{p} \mathbf{e} \bmod q$. This rounding error term $\mathbf{e}$ is a random variable with discrete, nearly uniform coefficients distributed over the interval $(-\frac{q}{2p}, \frac{q}{2p}]$.

Linearization [4] and BKW-style cryptanalysis [22] are ineffective here, as our sample complexity remains restricted to a factor of two relative to the lattice dimension. Furthermore, because the secret vectors $\mathbf{s}$ and $\mathbf{s}'$ possess sufficient density, they are resistant to the sparse-secret vulnerabilities identified by Albrecht [1]. Consequently, the security profile of this construction is primarily determined by the primal and dual attacks, both of which rely on the BKZ lattice reduction algorithm [33, 12].

The BKZ algorithm reduces a lattice or its basis using a shortest vector problem (SVP) oracle to a smaller b -dimensional lattice. Similar to other lattice-based KEMs, although the number of SVP oracle calls is a polynomial, we only consider the cost of one single call of the SVP oracle of lattice dimension b .

3.2.2 Weighted Primal Attack

The goal of the primal attack is to recover the secret vector $\mathbf{s} \in \mathbb{Z}_q^n$ and the noise vector $\mathbf{e} \in \mathbb{Z}_q^m$ given an LWE sample pair $(\mathbf{A}, \mathbf{b})$, where $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e} \bmod q$. Consider the following lattice Λ of dimension $d = m + n + 1$ and volume q^m defined by the set of vectors:

$$\Lambda = \{ \mathbf{v} \in \mathbb{Z}^{m+n+1} : (\mathbf{A} \mid \mathbf{I}_m \mid -\mathbf{b})\mathbf{v} \equiv 0 \pmod{q} \}$$

According to this lattice construction, the vector $\mathbf{v} = (\mathbf{s}, \mathbf{e}, 1)^\top$ is a member of Λ . Because the components of $\mathbf{s}$ and $\mathbf{e}$ are typically sampled from a small Gaussian distribution, this vector is significantly shorter than the expected length of a random vector in the lattice, which makes this vector the expected shortest vector. We then employ the BKZ (Block-Korkine-Zolotarev) reduction algorithm with block size b to find this vector. Here, the norm of this vector $\mathbf{v} = (\mathbf{s}, \mathbf{e}, 1)^\top$ is $\lambda \approx \sqrt{n\sigma_s^2 + m\sigma_e^2}$, where σ_s and σ_e are norms of $\mathbf{s}$ and $\mathbf{e}$ respectively. According to established heuristic models [3], the primal attack succeeds if the length of the shortest vector satisfies the following inequality:

$$\sqrt{n\sigma_s^2 + m\sigma_e^2} < \delta_b^{2b-d-1} \cdot \text{Vol}(\Lambda)^{\frac{1}{d}}$$

where, $\delta = \left(\frac{b}{2\pi e} (\pi b)^{1/b} \right)^{\frac{1}{2(b-1)}}$ is the root Hermite factor.

In this standard model [3], the assumption is that the norms $\mathbf{s}$ and $\mathbf{e}$ should be the same. However, in our case, $\|\mathbf{s}_i\| \neq \|\mathbf{e}_i\|$, where $\mathbf{s}_i, \mathbf{e}_i$ are the secret and error vectors of the i -th LWE samples. When $\|\mathbf{s}_i\| < \|\mathbf{e}_i\|$, the standard lattice construction is suboptimal. In paper [6], Bai and Galbraith introduce a lattice rescaling method which has been updated in the paper [13]. According to this updated rescaling method, the primal attack succeeds if $\sigma_s \sqrt{b} \leq \delta^{2b-d-1} \left(\frac{q}{\alpha} \right)^{\frac{m}{d}}$. Where: $\sigma_s \sqrt{b}$ is the projection of the target vector onto the last b vectors of the Gram-Schmidt orthogonalization. $\delta^{2b-d-1} \left(\frac{q}{\alpha} \right)^{\frac{m}{d}}$ represents the length of the $(d-b)$ -th Gram-Schmidt vector after the BKZ reduction.

3.2.3 Weighted Dual Attack

The goal of dual attack is to distinguish between a valid LWE sample $(\mathbf{A}, \mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e})$ and a sample chosen uniformly at random from the set $\mathbb{Z}_q^{m \times n} \times \mathbb{Z}_q^m$. Consider the following lattice

$$\Lambda = \{ (\mathbf{x}, \mathbf{y}) \in \mathbb{Z}^m \times \mathbb{Z}^n : \mathbf{A}^\top \mathbf{x} = \mathbf{y} \bmod q \}.$$

An adversary can distinguish these two samples if it can recover the shortest vector in the lattice Λ . Suppose we have the shortest vector $\mathbf{u} = (\mathbf{v}, \mathbf{w})$ of the lattice Λ . Then, compute a value $z = \mathbf{v}^\top \mathbf{b}$. If the sample is LWE sample, i.e., $\mathbf{b} = \mathbf{A}\mathbf{s} + \mathbf{e}$, then z will be $z = \mathbf{v}^\top (\mathbf{A}\mathbf{s} + \mathbf{e}) = (\mathbf{v}^\top \mathbf{A})\mathbf{s} + \mathbf{v}^\top \mathbf{e} = \mathbf{w}^\top \mathbf{s} + \mathbf{v}^\top \mathbf{e}$. Since $\mathbf{v}$ and $\mathbf{w}$ are short, and the LWE noise $\mathbf{e}$ and secret $\mathbf{s}$ are small, the resulting z will be small and follows a Gaussian-like distribution centred at zero. If the sample $(\mathbf{A}, \mathbf{b})$ is uniform, i.e., $\mathbf{b}$ is random, then $z = \mathbf{v}^\top \mathbf{b}$ will also be uniformly distributed modulo q . Therefore, observing the value of z , the adversary can distinguish these two samples.

If $\|\mathbf{s}\| < \|\mathbf{e}\|$, then the attacker can not easily find the shortest vector as in this case $\mathbf{w}^\top \mathbf{s}$ and $\mathbf{v}^\top \mathbf{e}$ are unbalanced. In this case, the weighted attack [6, 13] approach fixes this by scaling the error component by α . Consider the lattice $\Lambda' = \{(\mathbf{x}, \alpha\mathbf{y}) \in \Lambda\}$. By choosing the scaling factor α appropriately, the attacker balances the variance of $\mathbf{w}^\top \mathbf{s}$ and $\mathbf{v}^\top \mathbf{e}$.

The cost of the dual attack is measured by calculating the statistical distance $\epsilon = 4 \exp(-2\pi^2 \tau^2)$ between a uniformly distributed and a Gaussian distributed z , where $\tau = \frac{\|\mathbf{u}\| \sigma_{\mathbf{e}}}{q}$. Because the target key is hashed, a single statistical advantage (ϵ) is not sufficient to guarantee recovery. The attacker must repeat the process R times to achieve the necessary confidence, where R is determined by: $R = \max(1, \frac{1}{2^{0.2075 \cdot b \cdot \epsilon^2}})$. The total computational cost is the product of the BKZ lattice reduction effort and the required number of trials: $\text{Cost}_{\text{dual}} = \text{Cost}_{\text{BKZ}}(b) \cdot R$.

3.2.4 Core-SVP hardness of Scabbard

A simpler way to estimate the security of LWE/LWR-based schemes is using the Core-SVP hardness introduced in [3]. We use Dachman-Soled et al.'s state-of-the-art toolkit [15] to estimate SVP hardness of the underlying LWE-problem of our scheme. More detailed results are available in Table 8. In addition to Core-SVP hardness security, the estimated cost of classical attacks, based on [2], is also reported in Table 8 in terms of gates and bit memory requirements.

Table 8: Details of classical and quantum Core-SVP hardness of proposed Scabbard parameter sets. Beyond Core-SVP, the estimated cost of classical attacks in terms of gates and bit memory requirements.

Core-SVP, Primal attack	Scabbard-128	Scabbard-256	Scabbard-512
Lattice attack dimension b	1123	2165	3720
BKZ blocksize β	437	911	1762
Core-SVP classical hardness (CSS)	2^{128}	2^{266}	2^{514}
Core-SVP quantum hardness (QSS)	2^{116}	2^{241}	2^{467}
Beyond Core-SVP, classical attacks [2]			
Lattice attack dimension b'	1153	2305	4097
BKZ blocksize β'	446	938	1779
Gates	2^{161}	2^{300}	2^{534}
Memory in bits	2^{100}	2^{198}	2^{365}

3.2.5 Beyond core-SVP hardness

The core-SVP methodology provides an asymptotic, simplified lower bound for lattice reduction costs (e.g., $2^{0.292b}$ for classical adversaries). However, this metric is calculated assuming an idealized model that ignores significant real-world complexities and overheads hidden within the $o(b)$ term. To provide a highly realistic assessment of Scabbard's concrete security, we must look beyond the abstract core-SVP metric and address the specific mathematical realities of lattice sieving, attack asymmetries, and concrete gate-count estimations.

The Real-World Asymmetry between Primal and Dual Attacks : As shown in our analysis and other traditional core-SVP estimation methodology models, the dual attack is marginally cheaper or equivalent to the primal attack. In concrete cryptanalytic practice, however, the dual attack is significantly more computationally expensive as explained in [5]. The core-SVP estimation for the dual attack relies on a highly optimistic assumption that the sieving oracle in the first BKZ block

yields an exponentially large pool of vectors that all exactly match the length of the absolute shortest vector.

In reality, the geometry of lattice sieving dictates that while the algorithm returns many short vectors, the vast majority are geometrically longer than the absolute minimum by a factor of $\sqrt{4/3}$. For a dual attack to successfully construct an effective LWE distinguisher (requiring approximately $1/\epsilon^2$ short vectors), the adversary would need to compensate for this $\sqrt{4/3}$ length penalty by exponentially increasing the sieving iterations. This requirement fundamentally conflicts with modern "dimensions-for-free" optimizations. Conversely, the primal attack converts the LWE instance into a unique-SVP problem where finding a single exceptionally short target vector is sufficient. Consequently, the primal attack is universally the governing threat model that dictates the practical security ceiling of Scabbard. **Realistic Circuit, Gate-Count, and Memory Profiling of Sieving Oracles :** Rather than relying purely on abstract CPU cycle bounds, recent advancements analyze the precise gate costs required to instantiate the sieving oracle on explicit classical and quantum hardware architectures. By strictly profiling the exact number of logical gates required for XOR-popcount operations, hash evaluations, and inner-product checks across the geometric volume of the sieve, this methodology eliminates the ambiguity of the $o(b)$ factors.

Furthermore, even in these highly optimized gate-count estimations, the colossal memory overhead inherent to sieving is often ignored or abstracted away. In physical hardware, storing, sorting, and routing an exponentially growing database of lattice vectors imposes massive memory access latencies and energy bottlenecks that an adversary cannot bypass. When bounded mathematically by progressive sieving and Locality-Sensitive Hashing (LSH), and coupled with the inescapable realities of physical memory routing, the concrete cost demonstrates a substantial overhead above the theoretical core-SVP baseline. This explicit hardware-level accounting confirms that Scabbard possesses a comfortable margin of security beyond its stated core-SVP lower bound against both classical and quantum adversaries. An estimation of gate count and bit memory requirement following the methods described in the Kyber specification document [5] and Albrecht et al. [2] for different parameters of Scabbard is shown in Table 8.

3.2.6 Attacks using decryption failures

As with most lattice-based schemes based on variants of the LWE or LWR problems, the Scabbard KEM does not achieve perfectly correct decryption. There is a small probability of decryption failure. A chosen ciphertext adversary repeatedly performs decapsulation with chosen ciphertexts to find a valid ciphertext that fails due to the decryption failure probability. That event leaks information that is related to the long-term secret key $\mathbf{s}$. The adversary can recover the complete secret key by collecting sufficient such valid ciphertexts, and this type of attack is referred to as a decryption failure attack.

Attack complexity. To perform a decryption failure attack, the attacker needs to find several valid ciphertexts that cause decryption failure. The complexity to find one such ciphertext is directly proportional to $1/\delta$, where δ is the probability of decryption failure. The value of δ for each parameter of Scabbard is presented in Table 1. For most CCA-secure KEMs, an attacker is limited to at most 2^{64} queries to the decapsulation oracle [29, 5]. Even with 2^{64} queries, the probability of finding such a valid ciphertext that causes decryption failure is extremely low for all three parameter sets of Scabbard ($2^{64} \times 2^{-99} = 2^{-35}$ for Scabbard-128, 2^{-61} for Scabbard-256, and 2^{-61} for Scabbard-512). Additionally, multiple such valid ciphertexts are needed to recover the complete secret key. Thus, the decryption failure attack poses no practical threat to Scabbard.

3.3 Additional Security Properties

3.3.1 Forward Secrecy

Scabbard is key encapsulation mechanism designed to establish shared symmetric session keys. As with other lattice-based KEMs (e.g., ML-KEM/Kyber), Scabbard does not provide forward secrecy in isolation. Rather, it is a property of the higher-level key-establishment protocol in which Scabbard is deployed.

To achieve forward secrecy, Scabbard must be used in an ephemeral mode satisfying the following:

- **Ephemeral key generation:** For each new session, a fresh, ephemeral Scabbard key pair must be generated.

- **Authentication of ephemeral keys:** The ephemeral Scabbard public key must be authenticated, for example by binding it to a long-term signature key, or certificate.
- **Secure deletion of ephemeral secrets:** After decapsulation and session-key derivation, the ephemeral Scabbard secret key and any shared secrets must be securely erased from memory.
- **Separation from long-term credentials:** The protocol must ensure that long-term keys are kept separate from the ephemeral KEM keys. Future compromise of a long-term key should not reveal past session keys, provided that the corresponding ephemeral Scabbard private keys were discarded at the end of the session.

Therefore, Scabbard can support forward-secure communication when it is embedded in an authenticated key-agreement protocol that uses fresh ephemeral KEM keys and securely discards the corresponding secret material after each session. In this setting, the compromise of long-term credentials at a later time should not enable an adversary to recover previously established session keys.

3.3.2 Side-Channel Attack Resilience

Timing Attacks Long-term secrets can be extracted by observing timing differences, which are correlated to the secret material, in implementations of cryptographic schemes. An established countermeasure is the concept of constant-time code, in which no secret-dependent timing variations are present. All implementations provided for Scabbard are constant-time, which is easily achieved as Scabbard uses power-of-two moduli (no rejection sampling/modular reduction).

Differential Power and EM Analysis By simply passively observing the power consumption or electromagnetic emanations of a device execution cryptographic code, long-term secrets can be extracted by exploiting the correlation between the secret material and the physical behavior/leakage of the device. These attacks are powerful but require physical access to a device. They can be mitigated by hardening an implementation and explicitly integrating countermeasures, such as masking. [23, 11, 25] show that MLWR-based schemes perform much better than MLWE-based schemes when a masking countermeasure is integrated. It is due to the use of power-of-two moduli, which reduces the cost of two expensive masking components used in lattice-based KEMs: arithmetic-to-Boolean and Boolean-to-arithmetic conversions. The use of the rounding operation instead of the CBD-based error sampler also assists in reducing the masking cost. Therefore, we expect that the overall overhead of applying a masking countermeasure to Scabbard will be reasonable and thus practical in the context of real-world deployment.

3.3.3 Multi-key Attack Resilience

We analyze the security of Scabbard in a multi-target, or multi-user, setting, where an adversary is given access to T public keys and succeeds if it breaks the security of the KEM for any one of them. In this model, the adversary’s advantage is measured by its ability to distinguish or recover the shared secret associated with a valid encapsulation under any of the T target public keys. As a result, the overall attack advantage may increase with the number of targets, typically leading to a security loss that depends on T .

Public-key binding in encapsulation: To ensure that the security of Scabbard does not degrade significantly as the number of users T increases, we enforce strict domain separation through public key binding. During the encapsulation process, the random coins r used for the deterministic encryption of the message m are generated as:

$$(K, r) \leftarrow \text{XOF}(m \parallel \text{XOF}(\overline{pk}, \kappa), 2\kappa).$$

By hashing the public key pk and including it in the derivation of r , we ensure that the encryption process for every public key is cryptographically independent.

Multi-target security bound: Let $\text{Adv}_T^{\text{IND-CCA}}$ be the advantage of an adversary in breaking the IND-CCA2 security of Scabbard in a setting with T distinct public keys. Under the Random Oracle Model (ROM), we relate this to the security of a single target:

$$\text{Adv}_T^{\text{IND-CCA}} \approx T \cdot \text{Adv}_1^{\text{IND-CCA}}.$$

This linear degradation is the best achievable bound in the standard KEM setting. Because we bind pk to the internal coin generation and key derivation, the adversary cannot amortize the cost of the attack (e.g., via precomputed lattice reduction or rainbow tables) across different targets. Each target effectively acts as an independent random oracle instance.

Comparison to generic multi-key bounds: In generic constructions, the advantage of breaking a PRF or a KEM often degrades by a factor related to the number of evaluations or keys queried [8]. In *Scabbard*, due to the binding of pk at the point of generating the ephemeral coins r and the pre-shared key K , the adversarial advantage is constrained as follows:

- **No amortization:** An adversary cannot use ciphertexts intended for one target to gain information about another, as the re-encryption check in the FO transform is explicitly bound to the public key.
- **Deterministic generation:** The expansion of the matrix $\mathbf{A}$ from a seed, combined with the public key binding, ensures that the lattice instance itself is unique to each user. Even in a quantum setting, where Grover’s search might be applied, the uniqueness of the key-bound hash prevents the adversary from parallelizing the attack effectively across multiple users.

Consequently, *Scabbard* is designed to retain strong security in the multi-target setting. Even when an adversary is allowed to attack any one of T users, the cost of a successful attack remains governed primarily by the hardness of the underlying MLWE problem. Thus, large-scale deployment, such as in global IoT ecosystems, should not by itself cause a fundamental degradation of *Scabbard*’s security, beyond the expected quantitative loss associated with the number of targets.

3.3.4 (Temporary) Key Misuse Resilience

In practice, cryptographic implementations are frequently subjected to operational misuse, such as random number generator (RNG) failures, key reuse, or protocol-level identity confusion. *Scabbard* is designed to minimize the security impact of such failures by incorporating several security protocols directly into the key-derivation, encapsulation, and decapsulation phases.

Resilience against weak randomness (RNG Failures): The security of any public-key scheme relies on the quality of the deployed RNGs. *Scabbard* isolates the impact of RNG failures depending on the phase of the operation:

- **Encapsulation:** if the system RNG produces weak or predictable randomness (e.g., m) during encapsulation, the adversary can only compromise that specific session key. This is because the random seed r for *Scabbard*.PKE.Encrypt and session key K are derived deterministically via XOF with the encapsulation key $\overline{pk}$ (in *Scabbard*.KEM.Encaps). Therefore, a weak RNG during encapsulation can compromise the session keys, but cannot be exploited to extract the long-term secret key.
- **Decapsulation:** the decapsulation algorithm of *Scabbard* requires no fresh randomness. To handle invalid ciphertexts securely the decapsulation algorithm relies on the implicit rejection provided by the FO transformation. A pseudo-random byte string z is generated during KeyGen and stored within the secret key. If a ciphertext fails the re-encryption check, the decapsulation deterministically derives a rejection key using z and the ciphertext. This guarantees that the KEM never outputs a predictable error code, preventing timing or padding-oracle attacks even in poorly implemented protocols.

Public-key binding: to prevent protocol-level misuse, *Scabbard* binds the encapsulation process to the recipient’s exact public key. In *Scabbard*.KEM.Encaps(Algorithm. 13), the pre-shared key and the encryption coins are derived by hashing the random message m alongside the hash of the public key:

$$(K, r) \leftarrow \text{XOF}(m \parallel \text{XOF}(\overline{pk}, \kappa), 2\kappa).$$

By injecting $\text{XOF}(\overline{pk}, \kappa)$ into the derivation, the scheme achieves strict domain separation. If a protocol misuses the KEM—such as a developer accidentally encrypting the same message to multiple recipients,

or an attacker attempting to replay a ciphertext across different users, this derivation process ensures the resulting ciphertexts and internal states are completely uncorrelated.

API misuse prevention: To account for developer error in mixing up keys for different identities, the decapsulation key in **Scabbard** is constructed as $(\text{sk} \parallel \overline{\text{pk}} \parallel \text{XOF}(\text{pk}, \kappa) \parallel z)$. By caching the public key and its hash directly within the secret key array, the decapsulation routine has safe, zero-cost access to pk for the required re-encryption step. This prevents API misuse, as developers are not required to manually manage or pass the public key to the decapsulation function, eliminating a common source of protocol integration errors.

4 Advantages and Limitations

4.1 Advantages

- *Modularity*: implementations of **Scabbard** are extremely modular and parametrizable, allowing easy scaling to higher/lower security versions of the scheme.
- *No modular multiplication, no rejection sampling*: due to the choice of power-of-two moduli, modular reduction is implicit and free (performance wise), as well as rejection sampling. As a result, **Scabbard** implementations can be made constant-time relatively easily.
- *No error sampling and minimal pseudorandomness requirements*: due **Scabbard** relying on the MLWR hard problem, no error sampling (e.g. pseudorandomness) is required and simple rounding is used instead, which is efficient to implement. Especially in the masked domain, where sampling from a distribution becomes costly, **Scabbard** benefits performance-wise.
- *No fixed multiplication algorithm*: depending on optimization goal or target platform, **Scabbard** allows the implementer to choose an optimal polynomial multiplication algorithm. This translates to **Scabbard** achieving good performance on various platforms, including resource-constrained devices (Arm Cortex M4) and high-end 64-bit CPUs, as well as on hardware platforms (FPGA & ASIC), in which the inherent parallelism of certain multiplication algorithms can be exploited.
- *Efficient and easy to mask*: no masked modular reduction, efficient arithmetic to boolean conversions can be used for masking **Scabbard**, due to its choice of power-of-two moduli. As well as avoiding the need for costly masked sampling operations.
- *Low bandwidth*: MLWR constructions require less bandwidth than equivalent MLWE constructions.

4.2 Limitations

- *No NTT-based multiplication*: due to the use of power-of-two moduli, **Scabbard** cannot rely on the efficient and well-studied NTT algorithm for accelerating the polynomial multiplication without affecting the memory consumption. Different algorithms, such as Toom-Cook and Karatsuba multiplication, are required and are asymptotically slower compared to the NTT. However, as has been demonstrated in the literature, the freedom to choose a multiplication algorithm depending on a target platform can lead to improved performance, outperforming the NTT in some cases.

References

- [1] Martin R. Albrecht. On dual lattice attacks against small-secret lwe and parameter choices in helib and seal. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017*, pages 103–129, Cham, 2017. Springer International Publishing.
- [2] Martin R. Albrecht, Vlad Gheorghiu, Eamonn W. Postlethwaite, and John M. Schanck. Estimating quantum speedups for lattice sieves. In Shiho Moriai and Huaxiong Wang, editors, *Advances in Cryptology – ASIACRYPT 2020 – 26th International Conference on the Theory and Application of Cryptology and Information Security, Daejeon, South Korea, December 7–11, 2020, Proceedings, Part II*, volume 12492 of *Lecture Notes in Computer Science*, pages 583–613. Springer, 2020.
- [3] Erdem Alkim, Léoucas, Thomas Pöppelmann, and Peter Schwabe. Post-quantum key exchange – A new hope. In Thorsten Holz and Stefan Savage, editors, *25th USENIX Security Symposium, USENIX Security 16, Austin, TX, USA, August 10–12, 2016*, pages 327–343. USENIX Association, 2016.
- [4] Sanjeev Arora and Rong Ge. New algorithms for learning in presence of errors. In Luca Aceto, Monika Henzinger, and Jiří Sgall, editors, *Automata, Languages and Programming*, pages 403–415, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [5] Roberto Avanzi, Joppe Bos, Leoucas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehle. CRYSTALS-Kyber Algorithm Specifications And Supporting Documentation (version 3.02), 2021. <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf>.
- [6] Shi Bai and Steven D. Galbraith. Lattice decoding attacks on binary lwe. In Willy Susilo and Yi Mu, editors, *Information Security and Privacy*, pages 322–337, Cham, 2014. Springer International Publishing.
- [7] Abhishek Banerjee, Chris Peikert, and Alon Rosen. Pseudorandom functions and lattices. In David Pointcheval and Thomas Johansson, editors, *Advances in Cryptology – EUROCRYPT 2012*, pages 719–737, Berlin, Heidelberg, 2012. Springer Berlin Heidelberg.
- [8] Mihir Bellare, Thomas Ristenpart, and Stefano Tessaro. Multi-instance security and its application to password-based cryptography. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology – CRYPTO 2012 – 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2012. Proceedings*, volume 7417 of *Lecture Notes in Computer Science*, pages 312–329. Springer, 2012.
- [9] Jose Maria Bermudo Mera, Angshuman Karmakar, Suparna Kundu, and Ingrid Verbauwhede. Scabbard: a suite of efficient learning with rounding key-encapsulation mechanisms. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2021(4):474–509, Aug. 2021.
- [10] Joppe Bos, Leoucas, Eike Kiltz, T Lepoint, Vadim Lyubashevsky, John M. Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehle. Crystals – kyber: A cca-secure module-lattice-based kem. In *2018 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 353–367, 2018.
- [11] Olivier Bronchain and Gaëtan Cassiers. Bitslicing arithmetic/boolean masking conversions for fun and profit with application to lattice-based kems. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2022(4):553–588, 2022.
- [12] Yuanmi Chen and Phong Q. Nguyen. Bkz 2.0: Better lattice security estimates. In Dong Hoon Lee and Xiaoyun Wang, editors, *Advances in Cryptology – ASIACRYPT 2011*, pages 1–20, Berlin, Heidelberg, 2011. Springer Berlin Heidelberg.
- [13] Jung Hee Cheon, Duhyeon Kim, Joohee Lee, and Yongsoo Song. Lizard: Cut off the tail! A practical post-quantum public-key encryption from LWE and LWR. In Dario Catalano and Roberto De Prisco, editors, *Security and Cryptography for Networks – 11th International Conference, SCN 2018, Amalfi, Italy, September 5–7, 2018, Proceedings*, *Lecture Notes in Computer Science*, pages 160–177. Springer, 2018.

- [14] Stephen A. Cook and Stål O. Aanderaa. On the minimum computation time of functions. *Transactions of the American Mathematical Society*, 142:291–314, 1969.
- [15] Dana Dachman-Soled, Léo Ducas, Huijing Gong, and Mélissa Rossi. LWE with side information: Attacks and concrete security estimation. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology - CRYPTO 2020 - 40th Annual International Cryptology Conference, CRYPTO 2020, Santa Barbara, CA, USA, August 17-21, 2020, Proceedings, Part II*, Lecture Notes in Computer Science, pages 329–358. Springer, 2020.
- [16] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-lwr based key exchange, cpa-secure encryption and cca-secure kem. In Antoine Joux, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *Progress in Cryptology – AFRICACRYPT 2018*, pages 282–305, Cham, 2018. Springer International Publishing.
- [17] Eiichi Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael Wiener, editor, *Advances in Cryptology — CRYPTO’ 99*, pages 537–554, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg.
- [18] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. A modular analysis of the fujisaki-okamoto transformation. In Yael Kalai and Leonid Reyzin, editors, *Theory of Cryptography - 15th International Conference, TCC 2017, Baltimore, MD, USA, November 12-15, 2017, Proceedings, Part I*, Lecture Notes in Computer Science, pages 341–371. Springer, 2017.
- [19] Haodong Jiang, Zhenfeng Zhang, Long Chen, Hong Wang, and Zhi Ma. Post-quantum ind-cca-secure kem without additional hash. *IACR Cryptol. ePrint Arch.*, 2017:1096, 2017.
- [20] Matthias J. Kannwischer, Joost Rijneveld, and Peter Schwabe. Faster multiplication in $\mathbb{Z}_{2^m}[x]$ on cortex-m4 to speed up NIST PQC candidates. Cryptology ePrint Archive, Paper 2018/1018, 2018.
- [21] Anatolij A. Karatsuba and Yu. Ofman. Multiplication of multidigit numbers on automata. *Soviet physics. Doklady*, 7:595–596, 1963.
- [22] Paul Kirchner and Pierre-Alain Fouque. An improved bkz algorithm for lwe with applications to cryptography and lattices. In Rosario Gennaro and Matthew Robshaw, editors, *Advances in Cryptology – CRYPTO 2015*, pages 43–62, Berlin, Heidelberg, 2015. Springer Berlin Heidelberg.
- [23] Suparna Kundu, Jan-Pieter D’Anvers, Michiel Van Beirendonck, Angshuman Karmakar, and Ingrid Verbauwhede. Higher-order masked saber. In Clemente Galdi and Stanislaw Jarecki, editors, *Security and Cryptography for Networks - 13th International Conference, SCN 2022, Amalfi, Italy, September 12-14, 2022, Proceedings*, volume 13409 of *Lecture Notes in Computer Science*, pages 93–116. Springer, 2022.
- [24] Suparna Kundu, Archisman Ghosh, Angshuman Karmakar, Shreyas Sen, and Ingrid Verbauwhede. Rudraksh: A compact and lightweight post-quantum key-encapsulation mechanism. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(2):647–680, Mar. 2025.
- [25] Suparna Kundu, Angshuman Karmakar, and Ingrid Verbauwhede. On the masking-friendly designs for post-quantum cryptography. In Francesco Regazzoni, Bodhisatwa Mazumdar, and Sri Parameswaran, editors, *Security, Privacy, and Applied Cryptography Engineering - 13th International Conference, SPACE 2023, Roorkee, India, December 14-17, 2023, Proceedings*, volume 14412 of *Lecture Notes in Computer Science*, pages 162–184. Springer, 2023.
- [26] Suparna Kundu, Quinten Norga, Angshuman Karmakar, Shreya Gangopadhyay, Jose Maria Bermudo Mera, and Ingrid Verbauwhede. Scabbard: An exploratory study on hardware aware design choices of learning with rounding-based key encapsulation mechanisms. *ACM Trans. Embed. Comput. Syst.*, 24(1):10:1–10:40, 2025.
- [27] Adeline Langlois and Damien Stehlé. Worst-case to average-case reductions for module lattices. *Des. Codes Cryptography*, 75(3):565–599, June 2015.

- [28] Jose Maria Bermudo Mera, Angshuman Karmakar, Suparna Kundu, and Ingrid Verbauwhede. Scabbard: a suite of efficient learning with rounding key-encapsulation mechanisms. *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, 2021(4):474–509, 2021.
- [29] NIST. Submission Requirements and Evaluation Criteria for the Post-Quantum Cryptography Standardization Process, 2016. <https://csrc.nist.gov/CSRC/media/Projects/Post-Quantum-Cryptography/documents/call-for-proposals-final-dec-2016.pdf>.
- [30] Thales B. Paiva, Marcos Simplicio, Syed Hafiz, Bahattin Yildiz, Eduardo Cominetti, and Henrique Ogawa. Tailorable codes for lattice-based KEMs with applications to compact ML-KEM instantiations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025:139–163, 06 2025.
- [31] Prasanna Ravi, Debapriya Basu Roy, Shivam Bhasin, Anupam Chattopadhyay, and Debdeep Mukhopadhyay. Number ”not used” once - practical fault attack on pqm4 implementations of NIST candidates. In Ilia Polian and Marc Stöttinger, editors, *Constructive Side-Channel Analysis and Secure Design - 10th International Workshop, COSADE 2019, Darmstadt, Germany, April 3-5, 2019, Proceedings*, volume 11421 of *Lecture Notes in Computer Science*, pages 232–250. Springer, 2019.
- [32] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *J. ACM*, 56(6), September 2009.
- [33] Claus-Peter Schnorr and M. Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Math. Program.*, 66:181–199, 1994.